

计算机图形学期末复习资料

PPT + 学长/学姐笔记整合版
人话讲解 + 专业表述 + 典型例题

适用目标：期末应试、快速建立做题能力、背诵高频问答。

覆盖资料：L0-L8 PPT、计图笔记、汇总PDF、图片串讲资料。

使用建议：先看“考前路线图”和每章“考试怎么答”，最后刷“例题集”。

0. 考前路线图：这门课到底在考什么

一句话：计算机图形学就是“怎么用计算机把三维世界变成二维图像”。考试不会只考你背概念，更喜欢问：流水线每步干什么，OpenGL函数放在哪里，矩阵怎么乘，光照公式怎么解释，光线跟踪怎么写伪代码。

0.1 高频考点清单

优先级	考点	你要达到的程度
最高	OpenGL是什么、状态机、GL/GLU/GLUT、完整程序结构、常见函数	能写出程序框架，能解释 glBegin/glEnd、glutDisplayFunc、glutMainLoop、glMatrixMode、glLoadIdentity 等函数作用。
最高	流水线与顶点变换	能按“模型视图->投影->裁剪->透视除法->视口”说清楚，能算简单窗口坐标。
最高	平移/旋转/缩放矩阵、逆矩阵、复合变换、绕固定点旋转	会写矩阵，会说“右边先作用”，会写 $M = T(pf) R T(-pf)$ 。
最高	视图与投影	会解释 gluLookAt、glOrtho、glFrustum、gluPerspective；知道正交投影和透视投影优缺点。
最高	Z-buffer隐藏面消除	会说深度缓存怎么比较，OpenGL启用三步，和CullFace区别。
最高	Phong/Blinn光照、光源/材质、法向量、Gouraud/Phong着色	会写环境光+漫反射+镜面反射，能解释每个向量和指数。
高	光线跟踪、Whitted-style、伪代码、加速结构、抗锯齿	能讲“对每个像素发射光线，找最近交点，递归反射/折射/阴影”。
高	Bezier曲线/曲面、线性/双线性插值、齐次坐标	会写公式，会做 $t=0.5$ 这类小计算。
中高	SDF、点云、Blendshape、质点弹簧、拉普拉斯平滑、双目视觉、AIGC	偏简答题：会定义、会说算法步骤、会举用途。

0.2 复习顺序

- 先背框架：**成像四要素、流水线、OpenGL程序结构。
- 再练公式：**变换矩阵、视口变换、Phong、反射向量、Bezier。
- 最后背简答：**Z-buffer、光线跟踪、SDF/点云、Blendshape、质点弹簧、AIGC。

不要这样复习：不要从PPT第一页硬啃每张图。考试题通常不是问你“第几页是什么图”，而是把核心概念改写成：比较、解释、写公式、写步骤、写伪代码。

1. 图形学概述与成像系统

1.1 计算机图形学是什么

专业说法：计算机图形学研究利用计算机生成图像的方法，涉及硬件、软件和应用。

人话：你给电脑一些“场景信息”，比如物体长什么样、表面是什么材料、光从哪里来、相机在哪里，电脑算出最后屏幕上每个像素是什么颜色。

1.2 三大研究内容

内容	人话解释	例子
建模 Modeling	先把物体造出来，也就是“画什么”。	用三角网格表示人脸，用控制点生成 Bezier 曲线。
渲染 Rendering	把模型画成图，也就是“怎么画得像”。	光照、阴影、纹理、反射、折射。
动画 Animation	让物体动起来或变形，也就是“怎么连续变化”。	小球下落、布料飘动、人脸 Blendshape。

1.3 成像四要素

要素	含义	考试表述
几何 Geometry	物体的位置、形状、大小。	由点、线、多边形、曲面等图元构成。
材质 Material	物体表面怎么反光。	漫反射、镜面反射、折射、透明等。
光照 Light	照亮物体的光源。	环境光、点光源、方向光、聚光灯。
观察者 Viewer/Camera	从哪里看，用什么相机看。	视点、视线方向、投影方式、视景物。

考试怎么答

问“图形学如何生成图像”时，可以答：先建立场景的几何、材质、光源和观察者，再经过图形流水线或光线跟踪计算每个像素的颜色，最终写入帧缓存显示。

1.4 CG、CV、AI的关系

- **CG (Computer Graphics)**：创建图像，从数据/模型生成图。
- **CV (Computer Vision)**：解析图像，从图中理解信息。
- **AI**：可以作为CG和CV的工具。例如AIGC用深度学习生成图像、视频、3D资产。

1.5 颜色：RGB与CMY

系统	类型	人话	应用
----	----	----	----

RGB	加色系统	红绿蓝光加起来，越加越亮。	显示器、投影仪。
CMY	减色系统	从白光中吸收一部分颜色，越加越暗。	打印、照片负片。

1.6 针孔相机模型

针孔相机把三维点投影到二维成像平面。核心现象是：离相机越远，投影越小；这就是透视效果的来源。

相似三角形直观公式： $x' = d * x / z$ ， $y' = d * y / z$ 。这里 d 是投影平面到针孔的距离， z 越大，投影越小。

记忆：针孔模型不是为了造相机，而是为了让你理解“3D点怎么变成2D点”。后面的透视投影矩阵就是把这个过程写成矩阵乘法。

2. OpenGL与图形绘制流水线

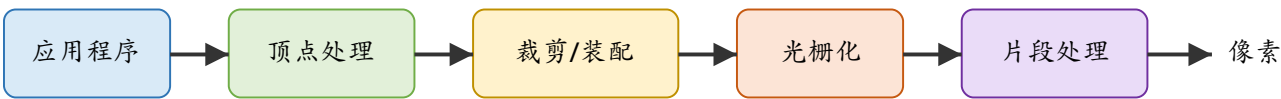
2.1 OpenGL是什么

标准答案：OpenGL是开放图形程序库，是一种应用程序编程接口（API），是图形硬件的软件接口，不是一种编程语言。

人话：OpenGL像一套“指挥显卡画图的命令”。你用C/C++调用这些函数，告诉它画什么、用什么颜色、相机怎么摆、开不开光照。

2.2 理解OpenGL的三个角度

角度	解释	考试关键词
API	程序员调用函数来绘图。	gl*、glu*、glut*
状态机	OpenGL内部有很多状态，函数会修改状态，后续绘制受状态影响。	当前颜色、当前矩阵、当前光源、当前材质
绘制流水线	顶点和图元按顺序经过一系列阶段变成像素。	顶点处理、裁剪、光栅化、片段处理、帧缓存



图形流水线：从顶点/图元到屏幕像素。

2.3 流水线各阶段

阶段	做什么	人话
应用程序	提交顶点、图元、状态。	你写代码把数据喂给OpenGL。
顶点处理	对顶点做模型视图变换、投影变换，计算顶点属性。	把物体从自己的坐标系搬到相机能看的坐标系。
裁剪与图元装配	把顶点组装成线/三角形/多边形，裁掉视景物体外部分。	看不见的先不要画。
光栅化	把几何图元转成片段。	三角形覆盖了哪些像素位置。
片段处理	计算颜色、深度测试、纹理等。	决定这个像素最终是什么颜色，是否被遮挡。
帧缓存	保存最终像素。	屏幕上要显示的那张图。

2.4 局部光照与全局光照

类型	代表方法	特点
局部光照	Phong模型、OpenGL传统固定管线	只考虑光源、物体表面、视点之间的局部关系，速度快，适合实时。
全局光照	Ray Tracing、Radiosity、Path Tracing	考虑反射、折射、阴影、间接光，效果真实但计算量大。

考试怎么答

如果问“为什么实时系统常用局部光照”，答：全局光照更真实，但要追踪大量光线或能量传递，计算量大；局部光照如Phong只在局部点上计算，便于硬件加速，适合交互式系统。

3. OpenGL程序结构与常见函数

3.1 一个OpenGL程序的基本骨架

- 1. 初始化GLUT和窗口：大小、位置、显示模式。
- 2. 初始化OpenGL状态：背景色、投影矩阵、模型视图矩阵、光照、深度测试等。
- 3. 注册回调函数：显示、窗口变化、键盘、鼠标、空闲函数。
- 4. 进入事件循环：glutMainLoop()。

```
#include <GL/glut.h>

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glBegin(GL_POLYGON);
        glVertex2f(-0.5f, -0.5f);
        glVertex2f( 0.5f, -0.5f);
        glVertex2f( 0.5f,  0.5f);
        glVertex2f(-0.5f,  0.5f);
    glEnd();
    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(500, 500);
    glutCreateWindow("simple");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

3.2 常见函数速查

函数	作用	考试注意
glutInit	初始化GLUT。	必须在其他GLUT函数前调用。
glutInitDisplayMode	设置显示模式。	如 GLUT_RGB GLUT_DOUBLE GLUT_DEPTH。
glutCreateWindow	创建窗口。	glutMainLoop前窗口不真正进入事件循环。
glutDisplayFunc	注册显示回调。	窗口需要重绘时调用display。
glutReshapeFunc	注册窗口大小改变回调。	常在里面设置glViewport和投影矩阵。
glutKeyboardFunc / glutMouseFunc	键盘/鼠标事件。	交互题常考。

glutIdleFunc	空闲时反复调用。	可用于动画更新。
glutPostRedisplay	请求重新绘制。	动画中更新变量后调用。
glClearColor	设置清屏颜色。	设置状态，不是立刻清屏。
glClear	清空缓冲。	常见：颜色缓冲+深度缓冲。
glBegin/glEnd	开始/结束图元定义。	中间用glVertex提交顶点。
glVertex*	指定顶点。	2d/3f等后缀表示参数类型和个数。
glColor*	设置当前颜色。	启用光照后可能被材质系统影响。
glMatrixMode	指定当前操作哪个矩阵栈。	GL_MODELVIEW、GL_PROJECTION、GL_TEXTURE。
glLoadIdentity	把当前矩阵变成单位矩阵。	防止上一次变换残留。
glTranslate/glRotate/glScale	修改当前矩阵。	影响后面提交的物体。
glPushMatrix/glPopMatrix	保存/恢复当前矩阵。	画多个物体时防止变换互相污染。
glEnable/glDisable	启用/关闭OpenGL功能。	如GL_DEPTH_TEST、GL_LIGHTING。

3.3 “当前矩阵”到底是什么

OpenGL有多个矩阵栈，但同一时刻只有一个矩阵栈处于“当前”状态。你用 `glMatrixMode(GL_PROJECTION)`，后面的 `glLoadIdentity`/`glOrtho`/`gluPerspective` 就修改投影矩阵；你用 `glMatrixMode(GL_MODELVIEW)`，后面的 `gluLookAt`/`glTranslate`/`glRotate` 就修改模型视图矩阵。

人话记忆：`glMatrixMode`像“切换编辑对象”。先选中哪张矩阵，后面那些变换命令就改哪张矩阵。

3.4 动画程序怎么写

动画的本质不是“一次画出运动”，而是反复刷新画面：更新变量 -> 重绘 -> 再更新 -> 再重绘。

```

float y = 1.0f;
float v = 0.0f;

void display() {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glPushMatrix();
        glTranslatef(0.0f, y, 0.0f);
        glutSolidSphere(0.1, 30, 30);
    glPopMatrix();
    glutSwapBuffers();
}

void idle() {
    v -= 0.0005f;    // 重力让速度变小
    y += v;          // 位置更新
    if (y < -1.0f) { // 简单反弹
        y = -1.0f;
        v = -v * 0.8f;
    }
    glutPostRedisplay();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutCreateWindow("falling ball");
    glEnable(GL_DEPTH_TEST);
    glutDisplayFunc(display);
    glutIdleFunc(idle);
    glutMainLoop();
}

```

3.5 窗口坐标与OpenGL坐标

OpenGL通常把窗口左下角当原点；很多窗口系统/鼠标坐标把左上角当原点。所以鼠标交互题要注意y方向可能相反。

考试怎么答

问“OpenGL为什么叫状态机”：OpenGL保存大量状态变量，如当前颜色、矩阵、光源、材质、是否启用深度测试。状态修改函数改变这些变量，后续绘制命令按照当前状态产生结果。

4. 三维显示、隐藏面消除与Z-buffer

4.1 在2D屏幕上制造3D效果靠什么

- **3D几何**：模型本身有三维坐标。
- **透视效果**：近大远小。
- **隐藏面消除**：被挡住的面不显示。
- **颜色与着色**：不同方向、材质、光照产生明暗。
- **光照、阴影、纹理、雾效**：增强真实感。

4.2 两类隐藏面消除

方法	核心思想	OpenGL函数	局限
背面剔除 CullFace	封闭物体的背面通常看不见，直接不画。	<code>glEnable(GL_CULL_FACE);</code> <code>glCullFace(GL_BACK);</code>	只适合剔除背向面，不能解决物体互相遮挡。
Z-buffer/ Depth Test	每个像素保存目前最近的深度，新片段更近才更新颜色。	<code>GL_DEPTH_TEST</code>	需要额外深度缓存，透明物体还要特殊处理。

4.3 Z-buffer算法

1. 给每个像素准备一个深度值，初始化为最远。
2. 每产生一个片段，就拿它的z值和该像素原来的z值比较。
3. 如果新片段更靠近相机，就更新颜色缓存和深度缓存。
4. 如果新片段更远，就丢弃。

伪代码：if new_depth < zbuffer[x,y] then color[x,y] = new_color; zbuffer[x,y] = new_depth; else discard。实际比较方向取决于深度定义，但本质就是“谁离相机近谁留下”。

4.4 OpenGL启用Z-buffer三步

1. 创建窗口时申请深度缓存：`glutInitDisplayMode(GLUT_RGB | GLUT_DOUBLE | GLUT_DEPTH)`
2. 初始化时启用深度测试：`glEnable(GL_DEPTH_TEST)`
3. 每帧绘制前清空深度缓存：`glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)`

例题：不开启深度测试会怎样？
答：如果不开启深度测试，OpenGL默认只按程序提交图元的顺序绘制。后画的物体可能覆盖先画的物体，即使后画的其实在远处。开启深度测试后，最终保留更靠近相机的片段。

考试怎么答

Z-buffer的关键点：每个像素一个深度缓存；绘制片段时比较深度；近的更新颜色和深度，远的丢弃；它解决投影方向上的遮挡问题。

5. 向量、矩阵与齐次坐标

5.1 向量基本运算

概念	公式	几何意义
模长	$ a = \sqrt{a_x^2 + a_y^2 + a_z^2}$	向量长度。
归一化	$a_hat = a / a $	方向不变，长度变成1。
点乘	$a \cdot b = a_x b_x + a_y b_y + a_z b_z = a b \cos(\theta)$	判断夹角、投影、光照强度。
叉乘	$a \times b$	得到同时垂直于a和b的向量，常用于求法向量。

5.2 点乘考试怎么用

- $a \cdot b > 0$: 夹角小于90度，大致同向。
- $a \cdot b = 0$: 垂直。
- $a \cdot b < 0$: 夹角大于90度，大致反向。
- 光照中: $\max(n \cdot l, 0)$ 表示光照方向和法向越接近，漫反射越强。

5.3 叉乘求三角形法向量

给三角形顶点 p_0, p_1, p_2 : $n = \text{normalize}((p_1 - p_0) \times (p_2 - p_0))$ 。方向由顶点顺序和右手法则决定。

例题：求法向量

$p_0=(0,0,0)$, $p_1=(1,0,0)$, $p_2=(0,1,0)$ 。

$p_1-p_0=(1,0,0)$, $p_2-p_0=(0,1,0)$, 叉乘为 $(0,0,1)$, 所以单位法向量 $n=(0,0,1)$ 。

5.4 矩阵乘法：为什么顺序重要

矩阵乘法一般不交换， AB 通常不等于 BA 。在图形学中，先旋转再平移，和先平移再旋转，结果完全不同。

考试常坑：如果点是列向量，写 $p' = M p$ ，那么复合矩阵 $M = T R S$ 的作用顺序是：先S缩放，再R旋转，最后T平移。右边最先作用。

5.5 齐次坐标

齐次坐标用N+1个数表示N维点。三维点通常写成 (x, y, z, w) 。

w值	意义	例子
----	----	----

w=1	表示点。	$(x,y,z,1)$
w=0	表示方向向量。	$(dx,dy,dz,0)$
w不为0	可以除以w回到普通坐标。	$(x,y,z,w) \rightarrow (x/w, y/w, z/w)$

为什么需要齐次坐标：平移不是普通3x3线性变换，但用4x4齐次矩阵后，平移、旋转、缩放、投影都能统一成矩阵乘法。

考试怎么答

齐次坐标的意义：用N+1维表示N维几何对象，使平移、投影等仿射/射影变换可以统一表示为矩阵乘法；w=1通常表示点，w=0表示方向向量。

6. 模型变换：平移、旋转、缩放、逆与复合

6.1 平移矩阵

$$T(dx, dy, dz) = \begin{bmatrix} 1 & 0 & 0 & dx \\ 0 & 1 & 0 & dy \\ 0 & 0 & 1 & dz \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

作用：把点整体移动。逆矩阵： $T^{-1}(dx, dy, dz) = T(-dx, -dy, -dz)$ 。

6.2 缩放矩阵

$$S(sx, sy, sz) = \begin{bmatrix} sx & 0 & 0 & 0 \\ 0 & sy & 0 & 0 \\ 0 & 0 & sz & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

逆矩阵： $S^{-1}(sx, sy, sz) = S(1/sx, 1/sy, 1/sz)$ ，前提是缩放因子不为0。

6.3 旋转矩阵

不同教材行列向量约定可能导致正负号位置不同。考试按老师PPT/课堂约定写。下面给常见列向量右手系版本：

$$\text{绕z轴逆时针旋转 } \theta: R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \text{ 绕x轴: } R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \text{ 绕y轴: } R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

旋转逆矩阵： $R^{-1}(\theta) = R(-\theta) = R^T(\theta)$ 。旋转矩阵是正交矩阵，所以逆等于转置。

6.4 复合变换

把多个变换矩阵乘成一个总矩阵，之后对大量顶点统一使用，效率更高。

例题：右边先作用

设点 $p=(1,1,1,1)^T$ ，先缩放 $S(2,2,2)$ ，再平移 $T(2,0,0)$ ，总矩阵写作 $M=T \cdot S$ 。

先 $S: p \rightarrow (2,2,2,1)$ 。再 $T: p' \rightarrow (4,2,2,1)$ 。

如果写成 $S \cdot T$ ，那就是先平移再缩放，结果不同。

6.5 绕固定点旋转

旋转中心必须先移到原点。绕固定点 pf 旋转的矩阵：

$$M = T(pf) \cdot R(\theta) \cdot T(-pf)$$

人话：先把旋转中心搬到原点，绕原点转，再把旋转中心搬回去。

例题：绕点(1,0)旋转90度

点 $P=(2,0)$ ，绕固定点 $pf=(1,0)$ 逆时针90度。

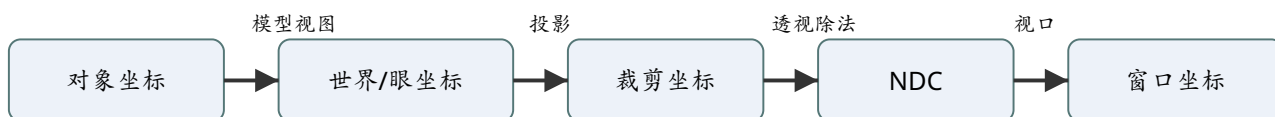
1. 先平移到以 pf 为原点： $P-pf=(1,0)$ 。
2. 旋转90度： $(1,0) \rightarrow (0,1)$ 。
3. 移回去： $(0,1)+pf=(1,1)$ 。

所以结果是 $(1,1)$ 。

考试怎么答

逆变换：平移取反，旋转取负角/转置，缩放取倒数；复合变换注意矩阵乘法顺序不交换；绕某点旋转公式 $M=T(pf)R T(-pf)$ 。

7. 相机、投影与视口变换



顶点变换总流程：考试常让你从输入点算到窗口坐标。

7.1 模型视图矩阵与投影矩阵

矩阵	负责什么	常用函数
模型视图矩阵 GL_MODELVIEW	描述物体和相机之间的相对位置、方向。	gluLookAt、glTranslate、glRotate、glScale
投影矩阵 GL_PROJECTION	描述相机内部参数和投影方式。	glOrtho、glFrustum、gluPerspective
视口变换	把规范化坐标映射到窗口像素坐标。	glViewport

7.2 gluLookAt

```
void gluLookAt(
    eyex, eyey, eyez,    // 相机位置 eye
    centerx, centery, centerz, // 相机看向的点 center/at
    upx, upy, upz        // 相机向上的参考方向 up
);
```

人话：第一组三个数是“眼睛在哪里”，第二组三个数是“眼睛盯着哪里”，第三组三个数是“头顶朝哪个方向”。

up向量不垂直怎么办

up不一定正好垂直于视线方向。实现时通常先用视线方向和up构造一个正交基：视线方向n、右方向u、真正向上方向v。核心做法是叉乘加归一化。

一种常见构造： $n = \text{normalize}(\text{eye} - \text{center})$ // 相机朝后的方向 $u = \text{normalize}(up \times n)$ // 相机右方向 $v = n \times u$ // 修正后的上方向

7.3 正交投影 glOrtho

```
glOrtho(left, right, bottom, top, near, far);
```

特点：投影线平行，没有近大远小，保持平行关系和尺寸比例，常用于CAD、工程三视图。

人话：正交投影像“远处用望远镜看”，物体远近不会改变大小。

7.4 透视投影 glFrustum / gluPerspective

```
glFrustum(left, right, bottom, top, near, far);  
gluPerspective(fovy, aspect, near, far);
```

特点：有近大远小，更符合人眼视觉，立体感强，但不保持距离和形状。

函数	特点
glFrustum	最通用，可以定义非对称视景体。
gluPerspective	更常用，用视角fovy、宽高比aspect、near/far定义对称透视体。

7.5 视口变换 glViewport

`glViewport(Ox, Oy, width, height)` NDC坐标 (x,y,z) in $[-1,1]$ $WinX = Ox + (x + 1) / 2 * width$ $WinY = Oy + (y + 1) / 2 * height$ $WinZ = (z + 1) / 2$

例题：NDC转窗口坐标

视口为 `glViewport(0, 0, 800, 600)`，NDC点为 $(0,0,0)$ 。

$WinX = 0 + (0+1)/2 * 800 = 400$ ； $WinY = 0 + (0+1)/2 * 600 = 300$ ； $WinZ = (0+1)/2 = 0.5$ 。

所以窗口坐标为 $(400, 300, 0.5)$ 。

考试怎么答

顶点变换流程：对象坐标经过模型视图矩阵到眼坐标，再经过投影矩阵到裁剪坐标，做透视除法得到规范化设备坐标，最后通过 `glViewport` 映射到窗口坐标。

8. 光照与明暗着色

8.1 为什么需要明暗着色

只有隐藏面消除时，物体像“平面色块”；明暗着色根据表面方向、光源位置、视点位置和材质属性计算颜色，让三维形状显得有体积感。

8.2 光源类型

光源	表示/特点	人话
环境光 Ambient	没有明确方向，场景中到处都有。	模拟多次反射后的底光。
点光源 Point Light	有位置，向四周发光。	灯泡。
方向光 Distant/Directional	只给方向，没有具体位置。	太阳光近似。
聚光灯 Spot Light	有位置、方向、角度范围。	手电筒。

8.3 表面类型

- **漫反射面**：粗糙表面，向各方向散射，看起来不闪。
- **镜面**：光滑表面，在反射方向附近形成高光。
- **透明面**：光会透过去，还会折射。

8.4 理想镜面反射方向

$r = 2(l \cdot n)n - l$ 。l、n、r通常都取单位向量；n是法向量，l是入射光方向，r是理想反射方向。

8.5 Phong光照模型

Phong模型把局部光照分成三项：环境光、漫反射、镜面反射。

$I = I_a k_a + I_l k_d \max(n \cdot l, 0) + I_l k_s \max(r \cdot v, 0)^{\text{shininess}}$
I_a: 环境光强度 I_l: 光源强度 k_a, k_d, k_s: 材质的环境/漫反射/镜面反射系数 n: 法向量, l: 光照方向, r: 理想反射方向, v: 视点方向
shininess: 高光指数，越大高光越小越锐利

人话理解

- **环境光**：不管方向，给物体一个基础亮度。
- **漫反射**：光越正对表面，越亮；看 $n \cdot l$ 。
- **镜面反射**：视线越靠近理想反射方向，高光越强；看 $r \cdot v$ 的指数。

8.6 Blinn-Phong模型

Blinn模型不用直接算r，而是使用半程向量 $h = \text{normalize}(l + v)$ ，镜面项常写为 $\max(n \cdot h, 0)^s$ 。它通常更方便计算。

8.7 平面着色、Gouraud着色、Phong着色

方法	怎么算	优缺点
Flat Shading 平面着色	每个多边形一个法向量，一个颜色。	快，但多边形感明显。
Gouraud Shading	顶点处算光照颜色，多边形内部插值颜色。	较平滑，但高光可能丢失。
Phong Shading	插值法向量，每个像素再算光照。	效果最好，计算较多。

8.8 OpenGL明暗处理步骤

1. 指定法向量：glNormal3f(...)。
2. 启用光照：glEnable(GL_LIGHTING)，启用某个光源如 glEnable(GL_LIGHT0)。
3. 设置光源属性和位置：glLightfv。
4. 设置材料属性：glMaterialfv。
5. 选择着色模式：glShadeModel(GL_SMOOTH) 或 GL_FLAT。

8.9 OpenGL光源位置的坑

重点：glLightfv(GL_POSITION,...) 会受到当时当前模型视图矩阵影响。也就是说，光源位置会被“记录在当前坐标状态下”。

- 在物体变换前设置光源：光源相对场景固定。
- 在某个物体变换后设置光源：光源可能跟着该物体/矩阵走。
- 在视图变换前设置光源：可表现为光源和视点一起移动。

例题：Phong简单计算

设环境项为0.1，漫反射系数和光强合并后为0.8， $n \cdot l = 0.5$ ，不考虑镜面项。颜色强度是多少？

$$I = 0.1 + 0.8 * 0.5 = 0.5。$$

解释：光不是正照表面，所以漫反射只贡献一半。

考试怎么答

Phong模型公式要会写，并解释三项：环境光给基础亮度，漫反射由法向和光照方向夹角决定，镜面反射由视线和理想反射方向夹角及高光指数决定。

9. 光线跟踪与Whitted-style Ray Tracing

9.1 流水线绘制 vs 光线跟踪

方法	循环对象	核心问题	特点
流水线/光栅化	for each object	这个物体覆盖哪些像素？	快，适合实时，需要Z-buffer处理深度。
光线跟踪	for each pixel	这个像素看到哪个最近物体？	真实感强，容易处理阴影、反射、折射，但计算量大。

9.2 光线跟踪基本思想

1. 从相机出发，通过每个像素发出一条主光线。
2. 求这条光线与场景物体的交点。
3. 选择离相机最近的交点。
4. 在交点处计算局部光照。
5. 为了阴影，向光源发射阴影光线，若被挡住则该光源不贡献直接光。
6. 为了镜面/透明效果，递归发射反射光线和折射光线。

```
for each pixel:
    ray = generateRay(camera, pixel)
    color[pixel] = trace(ray, depth=0)

trace(ray, depth):
    if depth > maxDepth:
        return background
    hit = nearestIntersection(ray, scene)
    if no hit:
        return background

    color = localPhong(hit)

    for each light:
        shadowRay = ray from hit point to light
        if shadowRay is blocked:
            remove/reduce this light contribution

    if material is reflective:
        color += ks * trace(reflectionRay, depth + 1)
    if material is transparent:
        color += kt * trace(refractionRay, depth + 1)
    return color
```

9.3 Whitted-style光线跟踪

Whitted-style是经典递归光线跟踪框架，核心是：主光线 + 阴影光线 + 反射光线 + 折射光线。它能自然产生阴影、镜面反射、透明折射等效果。

9.4 加速结构

复杂场景中，如果每条光线都和所有三角形求交，计算量会爆炸。所以需要空间加速结构。

结构	思想
均匀网格	把空间切成格子，光线只检查经过的格子里的物体。
BVH	用包围盒层次组织物体，先和大盒子求交，不交就跳过整组物体。
KD-tree	递归切分空间，减少无关求交。

9.5 抗锯齿：超采样

每个像素只打一条光线容易锯齿。超采样是在一个像素内部打多条光线，取平均颜色，边缘会更平滑。

例题：为什么光线跟踪不适合传统实时交互？

答：它要对每个像素发射光线，并与场景中物体求交；为了反射、折射、阴影还要递归发射更多光线，计算量大。因此传统上不适合实时交互。现代GPU/专用硬件能加速，使实时光线追踪逐渐可行。

考试怎么答

光线跟踪伪代码一定记：for each pixel -> generate ray -> nearest intersection -> local shading -> shadow ray -> recursive reflection/refraction。

10. 几何表示、Bezier、点云、SDF与网格处理

10.1 几何表示：显式与隐式

表示	定义	例子	优缺点
显式表示 Explicit	直接给出点的位置或参数方程。	$P=(1,2,3)$ ，曲线 $P(t)$ 。	容易生成点、容易绘制。
隐式表示 Implicit	不给具体点，给满足的方程/条件。	$x^2+y^2+z^2=1$ 。	容易判断内外/求交，但绘制可能要采样。

10.2 参数表示

参数曲线把点写成参数 t 的函数： $P(t)=(x(t),y(t),z(t))$ ，通常 t 在 $[0,1]$ 。

线段插值： $P(t) = (1-t)P_0 + tP_1$ ， $0 \leq t \leq 1$ 。

10.3 线性插值与双线性插值

线性插值：用两 endpoints 估计中间点。

$lerp(A,B,t) = (1-t)A + tB$

双线性插值：在矩形四个角上已知值，先沿 x 方向插两次，再沿 y 方向插一次。

已知 $Q_{11}, Q_{21}, Q_{12}, Q_{22}$ 。 $R_1 = lerp(Q_{11}, Q_{21}, t_x)$ $R_2 = lerp(Q_{12}, Q_{22}, t_x)$ $P = lerp(R_1, R_2, t_y)$

10.4 Bezier曲线

Bezier曲线由控制点控制形状，是一段 n 次多项式曲线。

n 次Bezier曲线： $B(t) = \sum_{i=0}^n C(n,i) (1-t)^{n-i} t^i P_i$ ， $0 \leq t \leq 1$ 一次： $B(t)=(1-t)P_0+tP_1$ 二次： $B(t)=(1-t)^2 P_0 + 2t(1-t)P_1 + t^2 P_2$

de Casteljau算法

反复在线段之间做线性插值。人话：控制点之间先连线，线段上按同一个 t 取点，再对新点继续线性插值，直到剩一个点，它就是曲线点。

例题：二次Bezier在 $t=0.5$ 处的点
 $P_0=(0,0)$ ， $P_1=(1,2)$ ， $P_2=(2,0)$ 。
 $B(0.5)=0.25P_0+0.5P_1+0.25P_2=(0,0)+(0.5,1)+(0.5,0)=(1,1)$ 。

10.5 Bezier曲面

曲面可以理解为“先按一个方向生成Bezier曲线，再按另一个方向把曲线上的点继续生成Bezier曲线”。例如3x3控制点，可先对每行按u生成三个点，再对这三个点按v生成曲面点。

10.6 点云

点云是空间中大量点的集合，每个点通常包含坐标，有时还有颜色、法向等属性。它常由三维扫描、双目视觉、激光雷达等得到。

10.7 SDF符号距离场

SDF (Signed Distance Field) 给空间中每个点一个数值：到表面的最近距离。符号表示内外：常见约定是内部为负，外部为正，表面为0。

转换	思路
点云 -> SDF	估计表面和法向，对空间采样点计算到点云/表面的距离，并判断内外符号。
SDF -> 点云/网格	提取SDF=0的等值面，例如用Marching Cubes，再采样得到点云。

10.8 网格与拉普拉斯平滑

网格通常由顶点、边、面组成。拉普拉斯平滑把一个顶点往邻居顶点的平均位置移动，用来去噪、让网格更光滑。

简单形式： $v_i' = v_i + \lambda(\text{average}(\text{neighbors of } v_i) - v_i)$ 。 λ 控制平滑强度。

注意：拉普拉斯平滑过强会让模型收缩、细节丢失。简答题可以提这个缺点。

考试怎么答

Bezier题背公式和de Casteljau；SDF题背“值=到表面距离，符号=内外，0等值面=表面”；拉普拉斯平滑题背“顶点向邻居平均位置移动”。

11. 动画与物理模拟：Blendshape和质点弹簧

11.1 Blendshape算法

Blendshape常用于人脸动画。它准备多个关键表情模型，然后用权重把它们线性混合成当前表情。

$$V = V_{base} + \sum_i w_i (V_i - V_{base})$$
，其中 w_i 是第*i*个表情的混合权重。

人话：基础脸 + 30%微笑 + 20%皱眉 + 10%张嘴 = 当前脸。

考试回答步骤

- 1. 准备基础模型和多个目标表情模型，保证拓扑一致、顶点一一对应。
- 2. 计算每个目标表情相对基础表情的位移。
- 3. 根据权重对位移线性组合。
- 4. 把结果加回基础模型，得到合成表情。

11.2 质点弹簧系统

质点弹簧系统用于模拟布料、软体等柔性物体。物体被离散成质点，质点之间用弹簧连接。

组成	含义
质点	有位置、速度、质量。
弹簧	连接质点，有初始长度和弹性系数。
力	弹簧力、阻尼力、重力、风力、碰撞力等。

基本计算流程

- 1. 初始化质点位置、速度、质量，初始化弹簧连接关系。
- 2. 对每根弹簧计算弹簧力：偏离原长越多，回复力越大。
- 3. 计算阻尼力，抑制无限振荡。
- 4. 累加外力，如重力、风力、碰撞。
- 5. 由牛顿第二定律 $a=F/m$ 得到加速度。
- 6. 用欧拉法或Verlet法更新速度和位置。
- 7. 循环迭代，得到动态效果。

弹簧力近似： $F_s = -k (current_length - rest_length) * direction$ 阻尼力近似： $F_d = -d * v$ 加速度： $a = F / m$ 显式欧拉： $v_{\{t+dt\}}=v_t+a dt$ ； $x_{\{t+dt\}}=x_t+v_{\{t+dt\}} dt$

数值稳定性：显式迭代步长太大、弹簧太硬时容易爆炸/抖动。可以减小时间步长、增加阻尼、使用更稳定的积分方法。

考试怎么答

质点弹簧系统答题关键词：柔性物体离散为质点和弹簧；根据弹簧力、阻尼力、外力算合力； $F=ma$ ；用欧拉/Verlet更新；步长过大可能数值不稳定。

12. 双目视觉、AIGC与其他高频简答

12.1 双目视觉与3D电影

双目视觉基于左右眼位置不同产生的视差。左右眼看到的图像略有差别，大脑根据对应点位移差异推断深度。

影响因素	说明
眼睛间距/相机基线	间距越大，视差越明显。
物体距离	越近视差越大，越远视差越小。
纹理与形状	特征越明显，越容易匹配左右图像对应点。

3D电影：给左右眼分别显示略有差别的图像，通过偏振/快门/红蓝等方式让每只眼只看到对应图像，从而产生立体感。

12.2 AIGC涉及哪些技术

- **深度学习：**神经网络是基础工具。
- **大语言模型：**文本理解、提示词、跨模态控制。
- **扩散模型/生成模型：**图像、视频、3D内容生成。
- **多模态分析与合成：**文本、图像、语音、视频、3D之间互相生成和理解。
- **计算机图形学：**渲染、几何表示、动画、物理模拟可参与生成内容的表达与控制。

12.3 纹理映射

纹理映射是在物体表面贴图，让模型不需要增加大量几何细节也能看起来复杂真实。核心是把纹理坐标(u,v)映射到模型表面，再在片段阶段取纹理颜色。

12.4 阴影

阴影表示某点到光源的路径被其他物体遮挡。光线跟踪中用阴影光线判断；光栅化中可用阴影贴图等方法。

12.5 透明与折射

透明材质不仅反射光，也透射光。光线跟踪可以递归生成折射光线；传统OpenGL中透明通常涉及混合和绘制顺序。

考试怎么答

这类简答题不要写玄学。按“定义->原理/步骤->用途/特点”三段答，就很稳。

13. 必背公式与代码速查

13.1 变换公式

主题	公式/结论
平移逆	$T^{-1}(dx, dy, dz) = T(-dx, -dy, -dz)$
旋转逆	$R^{-1}(\theta) = R(-\theta) = R^T(\theta)$
缩放逆	$S^{-1}(sx, sy, sz) = S(1/sx, 1/sy, 1/sz)$
绕固定点旋转	$M = T(pf)R(\theta)T(-pf)$
复合顺序	列向量约定下， $M = T R S$ ，则S先作用。
透视直观	$x' = d \cdot x / z$, $y' = d \cdot y / z$
视口变换	$WinX = Ox + (x+1)/2 * width$; $WinY = Oy + (y+1)/2 * height$; $WinZ = (z+1)/2$

13.2 光照公式

主题	公式/结论
三角形法向	$n = \text{normalize}((p1-p0) \times (p2-p0))$
理想反射方向	$r = 2(l \cdot n)n - l$
Phong	$I = I_a k_a + I_l k_d \max(n \cdot l, 0) + I_l k_s \max(r \cdot v, 0)^s$
Blinn	$h = \text{normalize}(l+v)$ ，镜面项 $\max(n \cdot h, 0)^s$

13.3 Bezier与插值

主题	公式/结论
线性插值	$\text{lerp}(A, B, t) = (1-t)A + tB$
n次Bezier	$B(t) = \sum C(n, i)(1-t)^{n-i}t^i P_i$
二次Bezier	$B(t) = (1-t)^2 P_0 + 2t(1-t)P_1 + t^2 P_2$

13.4 OpenGL启用功能速查

目标	关键代码
深度测试	<code>glutInitDisplayMode(... GLUT_DEPTH); glEnable(GL_DEPTH_TEST); glClear(... GL_DEPTH_BUFFER_BIT);</code>

光照	<code>glEnable(GL_LIGHTING); glEnable(GL_LIGHT0); glLightfv(...); glMaterialfv(...); glNormal3f(...);</code>
双缓冲	<code>glutInitDisplayMode(GLUT_DOUBLE ...); display中用glutSwapBuffers();</code>
矩阵设置	<code>glMatrixMode(GL_PROJECTION); glLoadIdentity(); 设置投影; 再 glMatrixMode(GL_MODELVIEW);</code>

14. 典型例题集：按考试题型训练

题型一：概念解释

1. 解释计算机图形学的主要研究内容。

答案：主要包括建模、绘制/渲染和动画。建模负责创建或重建三维物体；绘制/渲染负责根据光照、材质和观察者生成图像；动画负责生成运动或变形物体的连续图像，通常要考虑物理规律或运动规律。

2. OpenGL是什么？为什么说它是状态机？

答案：OpenGL是开放图形程序库，是图形硬件的软件接口和应用程序编程接口，不是编程语言。说它是状态机，是因为OpenGL内部保存当前颜色、矩阵、材质、光源、是否启用深度测试等状态；状态修改函数改变这些状态，后续绘图命令按当前状态执行。

3. 比较局部光照与全局光照。

答案：局部光照只考虑光源、表面点、视点之间的局部关系，例如Phong模型，计算量较小，适合实时绘制。全局光照考虑光在场景中的多次反射、折射、阴影、间接照明，例如光线跟踪和辐射度方法，效果更真实但计算量大。

题型二：OpenGL程序和函数

4. 写出OpenGL程序基本结构。

答案：main函数中先glutInit初始化；设置窗口大小、位置和显示模式；创建窗口；调用init设置OpenGL初始状态；注册display、reshape、keyboard、mouse、idle等回调函数；最后调用glutMainLoop进入事件处理循环。display函数中清缓冲、提交图元、刷新或交换缓冲。

5. glMatrixMode和glLoadIdentity的作用是什么？

答案：glMatrixMode用于指定当前操作的矩阵栈，如GL_PROJECTION表示之后修改投影矩阵，GL_MODELVIEW表示之后修改模型视图矩阵。glLoadIdentity把当前矩阵重置为单位矩阵，防止之前的变换残留影响新的设置。

题型三：变换计算

6. 说明绕固定点旋转的步骤和矩阵。

答案：若要绕固定点 pf 旋转，不能直接绕原点转。应先用 $T(-pf)$ 把固定点移到原点，执行旋转 $R(\theta)$ ，再用 $T(pf)$ 移回去。列向量约定下总矩阵为 $M=T(pf)R(\theta)T(-pf)$ 。

7. 已知 $\text{glViewport}(100,50,400,300)$ ，NDC点 $(0,0,0)$ ，求窗口坐标。

$\text{WinX}=100+(0+1)/2*400=300$ ； $\text{WinY}=50+(0+1)/2*300=200$ ； $\text{WinZ}=(0+1)/2=0.5$ 。所以窗口坐标为 $(300,200,0.5)$ 。

题型四：隐藏面与光照

8. Z-buffer算法原理是什么？

答案：Z-buffer为每个像素保存当前已绘制片段的深度值。绘制新片段时，把新片段深度与该像素的深度缓存比较；如果新片段更靠近相机，则更新颜色缓存和深度缓存，否则丢弃。这样最终可见的是离相机最近的表面。

9. 写出Phong光照模型并解释。

答案： $I = I_a k_a + I_l k_d \max(n \cdot l, 0) + I_l k_s \max(r \cdot v, 0)^s$ 。第一项是环境光，提供基础亮度；第二项是漫反射，取决于表面法向和光照方向夹角；第三项是镜面反射，取决于视线方向和理想反射方向夹角，s为高光指数，越大高光越集中。

10. Flat、Gouraud、Phong着色区别。

答案：Flat每个多边形一个颜色，速度快但不光滑；Gouraud在顶点计算颜色，再对内部插值，较平滑但可能丢高光；Phong对法向量插值，再对每个像素计算光照，效果更好但计算量更大。

题型五：光线跟踪与几何

11. 写出光线跟踪伪代码思路。

答案：对每个像素从相机发出主光线；求光线与场景最近交点；若无交点返回背景色；若有交点，则根据Phong模型计算局部光照；向光源发阴影光线判断是否被遮挡；若材质反射或透明，递归追踪反射光线和折射光线；把各部分贡献合成像素颜色。

12. 写出n次Bezier曲线公式。

答案： $B(t) = \sum_{i=0}^n C(n,i)(1-t)^{n-i}t^i P_i$ ， $0 \leq t \leq 1$ 。其中 P_i 是控制点， $C(n,i)$ 是组合数。也可以用de Casteljau算法通过反复线性插值构造。

13. 什么是SDF？如何与点云转换？

答案：SDF是符号距离场，空间中每个点的函数值表示它到物体表面的最近距离，符号表示在物体内部还是外，0等值面表示表面。点云转SDF需要估计表面/法向并计算采样点到表面的距离；SDF转点云或网格可以提取0等值面，再进行采样。

题型六：动画与模拟

14. Blendshape算法是什么？

答案：Blendshape通过对多个目标形状进行加权线性混合生成新形状，常用于人脸表情动画。公式可写为 $V = V_{base} + \sum w_i (V_i - V_{base})$ 。要求各目标模型拓扑一致、顶点对应一致。

15. 质点弹簧系统模拟步骤。

答案：将柔性物体离散为质点和弹簧；记录质点位置、速度、质量和弹簧原长、弹性系数；计算弹簧力、阻尼力、重力、风力等外力；由 $F=ma$ 得到加速度；用欧拉法或Verlet法更新速度和位置；循环迭代得到动态行为。显式方法步长过大可能不稳定。

15. 最后一页：考前背诵模板

15.1 万能简答结构

定义：它是什么。

原理/步骤：它怎么做。

特点/优缺点：为什么用它，有什么代价。

OpenGL/公式：能写函数或公式就补上。

15.2 最容易拿分的句子

- 计算机图形学研究利用计算机生成图像的方法，主要包括建模、渲染和动画。
- 成像四要素是几何、材质、光源和观察者。
- OpenGL是API，不是编程语言；它可以从API、状态机、绘制流水线三个角度理解。
- 图形流水线把顶点/图元经过顶点处理、裁剪装配、光栅化、片段处理，最终写入帧缓存。
- 模型视图矩阵描述物体与相机的相对位置，投影矩阵描述相机内部参数。
- 复合矩阵要注意乘法顺序，列向量约定下右边的变换先作用。
- Z-buffer为每个像素保存深度，只保留离相机最近的片段。
- Phong模型由环境光、漫反射和镜面反射组成。
- 光线跟踪是对每个像素发射光线，寻找最近交点，并递归处理阴影、反射和折射。
- Bezier曲线由控制点和Bernstein基函数定义，也可以用de Casteljau线性插值构造。

15.3 资料来源说明

本复习资料依据上传的计算机图形学课程PPT（Course Overview、Introduction、Image Formation、OpenGL Pipeline、OpenGL 3D and Interaction、Transformation 1/2、Shading 1/2、Ray Tracing、Geometry）、学长/学姐整理笔记、计算机图形学汇总PDF和图片串讲资料整理。内容按期末应试目标重组，保留必要专业术语，并增加人话解释与例题。

最后提醒：如果老师给了题型范围，优先按老师范围删减复习；如果没给范围，先背本资料中的“高频考点清单”和“必背公式与代码速查”。